

MaskSpec: Exactness-Preserving Compiler-Guided Scheduling for Structured Decoding

Dhanavanthesh S

Abstract

Structured generation alternates among deterministic literals, narrow grammar choices, recursive structures, and weakly constrained text. A single decoding strategy can therefore waste model or constraint-engine work: autoregressive decoding serializes forced regions, fixed-depth speculation overcommits where acceptance is poor, and byte-level fast-forwarding can silently change the model’s token history. MaskSpec is a compiler-guided runtime that selects at each decision epoch among locally constrained autoregressive decoding (AR), certified token-level jump-forward execution (JUMP), and locally constrained speculative decoding at a state-dependent depth (SPEC(K)). Its controller uses residual parser/compiler state and measured backend costs rather than a universal speculative depth.

MaskSpec’s organizing insight is a trust boundary between prediction and proof: a **ResidualDescriptor** estimates which mode may be profitable, whereas an **ExecutionCertificate** proves when a state transformation is admissible. Prediction errors affect performance; certificate errors affect semantics. We define exactness as equality to the stopped token-ID law of an explicit locally masked and renormalized constrained-AR process, not conditioning the unmasked model on eventual grammar membership. Under stated support, rollback, tokenizer, EOS, numerical, and adapted-scheduling assumptions, we formulate SPEC as a stopped-prefix kernel, give a singleton token-path certificate for JUMP, and show that history-dependent composition preserves the local reference law. A cost model charges certification, masking, target prefill, KV traffic, synchronization, and controller overhead, while a pre-registered evaluation makes the proposal falsifiable. MaskSpec is proposed, not implemented or evaluated, and this paper reports no performance results.

Document type: Pre-registered systems research proposal

1. Introduction

Constrained decoding and speculative decoding optimize different parts of language-model generation. A constraint engine identifies token IDs admitted at the current parser state. A speculative decoder changes how many positions a target-model evaluation can verify, while an acceptance-and-correction rule can preserve a declared target distribution Leviathan et al. 2023, Sec. 2.3; Chen et al. 2023, Sec. 2. Their combination is attractive because structured outputs move repeatedly between fixed syntax, short alternatives, recursive structure, and open text. Those regions plausibly differ in mask construction, draft acceptance, rollback, and target execution cost.

The reference distribution must be fixed before performance can be discussed. Local constrained decoding masks operationally illegal token IDs at each prefix and renormalizes. Global grammar conditioning instead restricts the complete-sequence law induced by an unmasked distribution p to outputs in a language L . The latter depends on future-validity probability $\Phi(hx) = \Pr_p[\text{eventual valid completion} \mid hx]$. If two locally admitted tokens a, b each have probability $1/2$, but $\Phi(ha) = 1$ and $\Phi(hb) = 1/10$, local masking samples them equally while global conditioning assigns probabilities $10/11$ and $1/11$. Grammatical validity therefore establishes neither distributional fidelity nor equality between these laws. MaskSpec preserves the former, operational law; its scheduling contribution is orthogonal to applications that require the latter.

A second distinction separates forced bytes from forced model tokens. Suppose an accepted byte string is **ab**, while the vocabulary contains $\tau_{ab} \mapsto \mathbf{ab}$, $\tau_a \mapsto \mathbf{a}$, and $\tau_b \mapsto \mathbf{b}$. If the local reference assigns probabilities 0.4 to τ_{ab} and 0.6 to τ_a , both paths may emit **ab**, yet they have different token probabilities and target hidden states. Injecting τ_{ab} removes a path of mass 0.6 . Token aliases, alternative segmentations, boundary-crossing tokens, UTF-8 fragments, normalization, EOS, and special tokens create related failures. A sound token-law JUMP therefore needs a token-path certificate, not a likely byte prefix. Even certified tokens normally must advance target history and KV state; batching can remove serial iterations but not target computation.

These observations motivate MaskSpec’s central separation. A **ResidualDescriptor** contains fallible performance features, such as parser route, syntactic region, branching, legal-set density, and estimated rollback work. An **ExecutionCertificate** contains the state-dependent evidence required to authorize an exact transformation. The scheduler may misrank AR, JUMP, and SPEC without changing the output law, provided every selectable kernel satisfies the same semantic contract and AR remains the fallback. Confidence never substitutes for a certificate.

This paper asks three questions. **RQ1:** under what conditions do constrained SPEC, certified JUMP, and their adaptive composition preserve the locally constrained target-token law? **RQ2:** can residual compiler state predict the best action beyond mask density and model-only signals after feature cost is charged? **RQ3:** when do JUMP or SPEC(K) improve end-to-end latency after target prefill, KV traffic, masking, rollback, controller, and synchronization costs are included?

The proposed contribution is the combination of (i) an explicit local reference semantics and stopped-prefix kernel contract; (ii) a trust boundary between heuristic residual descriptors and exact singleton token-path certificates; (iii) joint parser/model transaction semantics for AR, JUMP, and SPEC; and (iv) a falsifiable, backend-conditioned policy and evaluation protocol. The individual ingredients, including speculative sampling, parser-state features, token-aligned masks, forced execution, rollback, and adaptive depth, have prior art. The claim is about their integration and formal scoping, not priority for those mechanisms.

2. Background and Closest Prior Work

Speculative decoding. Exact speculative sampling draws from a cheaper proposal and uses target verification plus residual correction to reproduce a target law Leviathan et al. 2023; Chen et al. 2023. Sequoia studies acceptance-aware, hardware-aware tree speculation Chen et al. 2024. Dynamic lookahead and online policies are also established: DISCO uses draft confidence, SpecDec++ learns stopping decisions, and BanditSpec treats configuration choice as a bandit problem Mamou et al. 2024; Huang et al. 2025; Hou et al. 2025. MaskSpec cannot claim adaptive depth by itself; it asks whether constraint-derived state improves mode and depth selection while every chosen kernel targets the same operational law.

Grammar-constrained execution. Outlines compiles constraints into token filtering Willard and Louf 2023. XGrammar aligns compiled grammar execution with tokenizer vocabulary and exposes speculative masks, rollback, fork, and jump-forward primitives Dong et al. 2025. llguidance likewise supports fast-forwarding and rollback while documenting token-boundary hazards. GreatGrammar constructs tokenizer/lexer alignment tables and a pushdown-automaton path Park, Zhou, and D’Antoni 2025. Parser Stack Classification and Trie Automata show that parser configuration and constraint class can be compiled into cheaper mask selection Li et al. 2026; Xu and Bouyarmane 2026. These systems narrow any claim that residual parser state, tokenizer alignment, or mask specialization is independently new.

DOMINO constrained generation, *Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation*, builds vocabulary-aligned subterminal tries and predicts subsequent subterminals from scanner and parser context Beurer-Kellner, Fischer, and Vechev 2024, Secs. 3.2–3.6. Its configured bound controls how far this continuation mechanism looks; it does not document an online choice among AR, token-certified JUMP, and several SPEC depths. Its minimally invasive result preserves support rather than equality with MaskSpec’s locally renormalized law. GRID is closer to JUMP: it keys masks on lexer/LALR configurations, extends singleton masks into bounded forced spans, and uses persistent parser states Arjmandi 2026, Secs. 3–6. It does not combine exact draft-model speculation with residual-state profitability selection or a joint target-KV transaction theorem.

Local versus global semantics. Grammar-Aligned Decoding analyzes distortion introduced by local masking and estimates future correction Park et al. 2024. Future Validity formalizes the distinction and shows that a Leviathan-style verifier over local masks samples the locally projected law, not the globally grammar-conditioned law Nie et al. 2026, Defs. 1–2 and Prop. 3. *The Parser Already Knows* predicts approximate future-validity corrections from parser, lexer, and candidate-token state Özgü et al. 2026; this is direct prior art for parser state as predictive information, though its objective changes probabilities rather than execution mode. Adaptive Weighted Rejection Sampling (AWRS) gives an exact Las Vegas sampler for a locally constrained next-token law using black-box membership queries and estimates normalizers for approximate global correction Lipkin et al. 2025. It is a principled additional action or baseline where the operational constraint can be queried cheaply.

Table 1 is split into two portrait-width panels so that capability evidence remains readable without a rotated page. A categorical entry describes only what the cited primary sources document. “Not documented” is not evidence that a capability is absent.

Table 1a. Documented execution capabilities in the closest system families.

System or family	Tokenizer-aware constraint execution	Adaptive speculative depth	Forced multi-token execution	Exact execution certificate
Exact and adaptive speculative decoding [Leviathan et al. 2023; Mamou et al. 2024; Huang et al. 2025; Hou et al. 2025]	No	Partial	No	No
DOMINO constrained generation [Beurer-Kellner et al. 2024, Secs. 3.2-3.6]	Yes	Not documented in inspected sources	Partial	No
XGrammar and llguidance [Dong et al. 2025; pinned software evidence below]	Yes	Not documented in inspected sources	Partial	Partial
Parser-state mask compilers [Park et al. 2025; Li et al. 2026; Xu and Bouyarmane 2026]	Yes	No	Partial	Partial
Future Validity and parser correction [Nie et al. 2026; Özgü et al. 2026]	Partial	Not documented in inspected sources	No	No
GRID [Arjmandi 2026, Secs. 3-6]	Yes	No	Yes	Partial
MaskSpec	Not evaluated	Not evaluated	Not evaluated	Not evaluated

Table 1b. Documented state, scheduling, and systems contracts.

System or family	Parser/compiler predictors	Parser and target-KV rollback	Mode selection	Full-cost scope	Evaluation status
Exact and adaptive speculative decoding	Partial	Partial	No	Partial	Yes
DOMINO constrained generation	Yes	Not documented in inspected sources	No	Partial	Yes
XGrammar and llguidance	Partial	Partial	No	Partial	Yes
Parser-state mask compilers	Yes	No	No	Partial	Yes
Future Validity and parser correction	Yes	Partial	No	Partial	Yes
GRID	Yes	Partial	No	Partial	Yes
MaskSpec	Not evaluated	Not evaluated	Not evaluated	Not evaluated	Not evaluated

The rows necessarily aggregate systems with different reference laws and constraint formalisms. Exact speculative decoders reproduce their declared target law; DOMINO states a minimally invasive support objective; XGrammar,

llguidance, the mask compilers, and GRID implement operational constraint execution; Future Validity separates local projection from global grammar conditioning; MaskSpec targets the local operational token-ID law defined in Section 3. Cells marked “Partial” indicate that a related mechanism exists but not the complete MaskSpec contract. MaskSpec’s “Not evaluated” cells distinguish a design intention from demonstrated capability.

To our knowledge, among the primary sources inspected through 13 September 2026, we found no documented system combining state-dependent profitability prediction, independently validated token-path certificates, and online selection among AR, certified JUMP, and multiple exact constrained-SPEC depths under one local reference-law contract with joint parser/model transactions and critical-path cost accounting. This bounded claim concerns the integration contract, not priority for any individual mechanism.

3. Reference Semantics and Problem Definition

Let V be a finite token-ID vocabulary with decoding map d , explicit EOS and special-token semantics, committed history $h_t = x_{<t}$, complete constraint state s_t , and transition $\delta(s, x)$ on a token’s exact bytes and semantics. Raw target and draft logits are ℓ_p, ℓ_q ; p, q denote distributions after all declared non-constraint transformations. The controller acts only at decision epochs T_n and chooses $A_n \in \{\text{AR}, \text{JUMP}(r), \text{SPEC}(K)\}$.

Symbol	Meaning
V, d	token-ID vocabulary and decoding map, including special-token semantics
$h_t = x_{<t}$	committed target-model history
s_t, δ	complete constraint state and token transition
p, q	target and draft distributions after declared non-constraint processing
$\mathcal{A}_{\text{impl}}$	token IDs admitted by the operational session mask
$\mathcal{A}_{\text{viable}}$	token IDs with at least one accepting continuation
$p_{\text{local}}, q_{\text{local}}$	locally masked and renormalized target and draft laws
$\mathcal{H}_t, \mathcal{F}_t$	analysis and controller-visible filtrations
T_n, L_n, A_n	decision epoch, committed block length, and chosen action
$G(a \mid o), C(a \mid o)$	conditional progress and measured cost for observation o

The reference sampling pipeline is fixed:

```
target logits
-> declared constraint-independent logits processors
-> temperature and declared truncation
-> normalization to p
-> operational constraint mask
-> renormalization to p_local
-> sample.
```

Thus p need not be raw softmax. Processor order, parameters, tie-breaking, and history state are part of the reference configuration. Moving top- k or top- p after the mask generally changes the law. Stateful or randomized processors must add their state and randomness to the semantic transaction and filtration. AR, JUMP, and SPEC use the same target pipeline at the same committed history; q is defined analogously for the draft and may use different non-constraint processors.

Operational admission and ideal future viability are distinct:

$$\mathcal{A}_{\text{impl}}(s) = \{x \in V : \text{PublicSessionMask}(s)[x] = 1\},$$

$$\mathcal{A}_{\text{viable}}(s) = \{x \in V : \exists z \in V^* \text{ such that } \delta^*(s, xz) \text{ is accepting}\}.$$

MaskSpec uses $\mathcal{A}_{\text{impl}}$, the observable operational mask including the session-level EOS decision. It does not assume equality with $\mathcal{A}_{\text{viable}}$: an operationally admitted prefix can later dead-end, and a defective or approximate engine can omit a viable token. Language acceptance, operational-mask equality, and global distributional fidelity are separate properties.

Define

$$Z_p(h, s) = \sum_{y \in V} p(y \mid h) \mathbf{1}[y \in \mathcal{A}_{\text{impl}}(s)],$$

and, only when $Z_p(h, s) > 0$,

$$p_{\text{local}}(x \mid h, s) = \frac{p(x \mid h) \mathbf{1}[x \in \mathcal{A}_{\text{impl}}(s)]}{Z_p(h, s)}.$$

Define q_{local} analogously when $Z_q > 0$. If $Z_p = 0$, reference AR returns a typed failure; unmasking would define a different process. If $Z_q = 0$ while $Z_p > 0$, SPEC is unavailable. A singleton operational mask does not imply positive mass after prior truncation. EOS is legal only when both constraint and runtime contracts permit it, and no token follows EOS.

For comparison, let

$$L_V = \{x_{1:T} \in V^* : d(x_{1:T}) \text{ is accepted and terminates under the EOS policy}\}.$$

Because d need not be injective, equal byte strings can arise from different token histories. With

$$\Phi(hx) = \Pr_p[\text{eventual completion in } L_V \mid hx],$$

the globally conditioned next-token law, when its conditioning event has positive probability, is

$$p_{\text{global}}(x \mid h, \text{eventual } L_V) = \frac{p(x \mid h) \Phi(hx)}{\sum_y p(y \mid h) \Phi(hy)}.$$

MaskSpec targets equality of the stopped finite **token-ID sequence law** produced by repeated sampling from p_{local} , including EOS or typed failure. Equality of token laws implies equality of emitted-byte laws under d , but the converse fails when d is noninjective. Greedy equality and validity are weaker. Distributional equality also does not promise identical same-seed paths, because speculative execution consumes auxiliary randomness; pathwise reproducibility needs a separate counter-based RNG contract.

Let $\mathcal{H}_t$ be the analysis filtration containing committed reference tokens, semantic states, and auxiliary randomness revealed through token position t . The controller-visible $\mathcal{F}_t \subseteq \mathcal{H}_t$ contains only committed history, constraint and controller state, calibrated past measurements, prior actions, and randomness already exposed to the controller. At a decision epoch, a charged certificate probe may produce $\mathcal{G}_n \subseteq \mathcal{H}_{T_n}$, but $\mathcal{G}_n$ contains no future proposal, acceptance, correction, or target-sampling draw.

4. MaskSpec Design

MaskSpec is grounded in MaskForge rather than a disconnected parser. Read-only inspection of revision 9426a46 found regular, structured, and hybrid compile-time routes; structured and hybrid sessions share the structured runtime representation, while regular constraints use an automaton matcher. The public session exposes masking, one-token advance, reset, accepting/dead-state queries, and EOS lifecycle behavior, but no public checkpoint interface. Structured advancement already uses internal undo operations to make candidate-token processing atomic. This supports a transaction proposal, not a claim that cross-route rollback or joint model-KV transactions already exist.

4.1 Predictive State and Certificate State

A versioned **ResidualDescriptor** groups predictive information by provenance and availability:

Feature class	Examples	Earliest availability	Accounting rule
Static compiler	constraint family, schema region, automaton or parser-state class, nullable and branching structure, recursion or reference structure, precomputed singleton transitions, forced-run upper bound	After compilation and state identification	Charge compilation or amortize it over declared reuse
Dynamic parser	stack depth, active branches, validator cursors, accepting state, retained bytes, current legal-set size	During incremental parsing; exact legal-set size may require mask construction	Charge traversal and mask work before crediting the feature
Runtime and system	batch, backend, target/draft pair, recent acceptance, measured mask time, KV layout, device utilization	At request time or after prior actions	Charge profiling, synchronization, and controller access

This taxonomy gives “compiler-guided” a testable meaning: static compiler structure must provide predictive value or reusable certificate work beyond dynamic parser and runtime signals. If the useful signal is predominantly dynamic, the accurate empirical description is **constraint-state-guided scheduling**. Every field has a production cost and availability epoch. A policy cannot claim that exact mask density avoids mask construction if it first materializes the mask to obtain that density. A selected kernel may reuse a charged artifact, but the accounting must not erase its acquisition cost.

An **ExecutionCertificate** authorizes one exact action. The conservative JUMP certificate contains token path $(\tau_1, \dots, \tau_r)$, witnessed transitions, exact start-state identity, grammar/compiler/tokenizer and processor versions, special-token/EOS facts, a length cap, and a generation tag. Its decisive condition is

$$\mathcal{A}_{\text{impl}}(s_i) = \{\tau_i\}, \quad s_{i+1} = \delta(s_i, \tau_i), \quad i = 1, \dots, r.$$

Aliases, alternative segmentation, boundary-crossing tokens, UTF-8 fragments, normalization differences, EOS ambiguity, or a stale state invalidate the certificate. A finite hash may index a cache but cannot by itself prove exact state equality without a collision-free representation argument. A broader byte-equivalence certificate would need to show that every admitted tokenization induces the same future target and constraint distribution; equal decoded bytes are insufficient.

Certificate construction may precompute singleton regions for regular constraints, traverse a vocabulary trie at runtime, or conservatively build a full operational mask at each tentative step. The latter costs

$$C_{\text{cert}}(r) = \sum_{i=1}^r (C_{\text{mask}}(s_i) + C_{\text{advance}}(s_i, \tau_i)) + C_{\text{validation}},$$

and can approach serial constraint-decoding cost. Certificates are capped at R_{max} and invalidated by any version, state, generation, processor, or EOS mismatch. Coverage and amortization are empirical questions.

4.2 Joint Transactions and Controller

The proposed interface is:

```
mark = session.checkpoint()
session.advance(candidate)
session.rollback(mark)
session.commit_checkpoint(mark)
```

Handles are opaque, session-bound, generation-tagged, and initially strict-LIFO. Reset invalidates old handles; rollback cannot cross a commit; invalid, foreign, stale, or committed handles fail without mutation. Resource limits bound nesting and retained data. Regular routes should use constant-size snapshots where possible; structured routes should first expose and harden the existing undo log.

Parser state, target history/KV, and any stateful draft history/KV form one logical transaction. A rejection restores every component to the accepted prefix and discards every tentative suffix; a commit publishes their states together. A stateless or recomputed drafter changes the cost, not the semantic obligation. Rollback equivalence is observational: masks, accepted bytes, lifecycle and error state, descriptor, handle validity, and every future transition must equal replay from the checkpoint. Monotonic telemetry may remain external only if it cannot affect semantics and enters the controller filtration before use.

At decision epoch T_n , the controller obtains charged descriptor fields, decides whether to attempt a certificate, forms the eligible action set, and selects AR, a valid $\text{JUMP}(r)$, or $\text{SPEC}(K)$ for K in a backend-specific finite set. The probe result and charged cost may inform selection because they reveal current state, not future sampling randomness. An inapplicable action falls back to AR only after the joint state is shown unchanged. A prediction never authorizes JUMP.

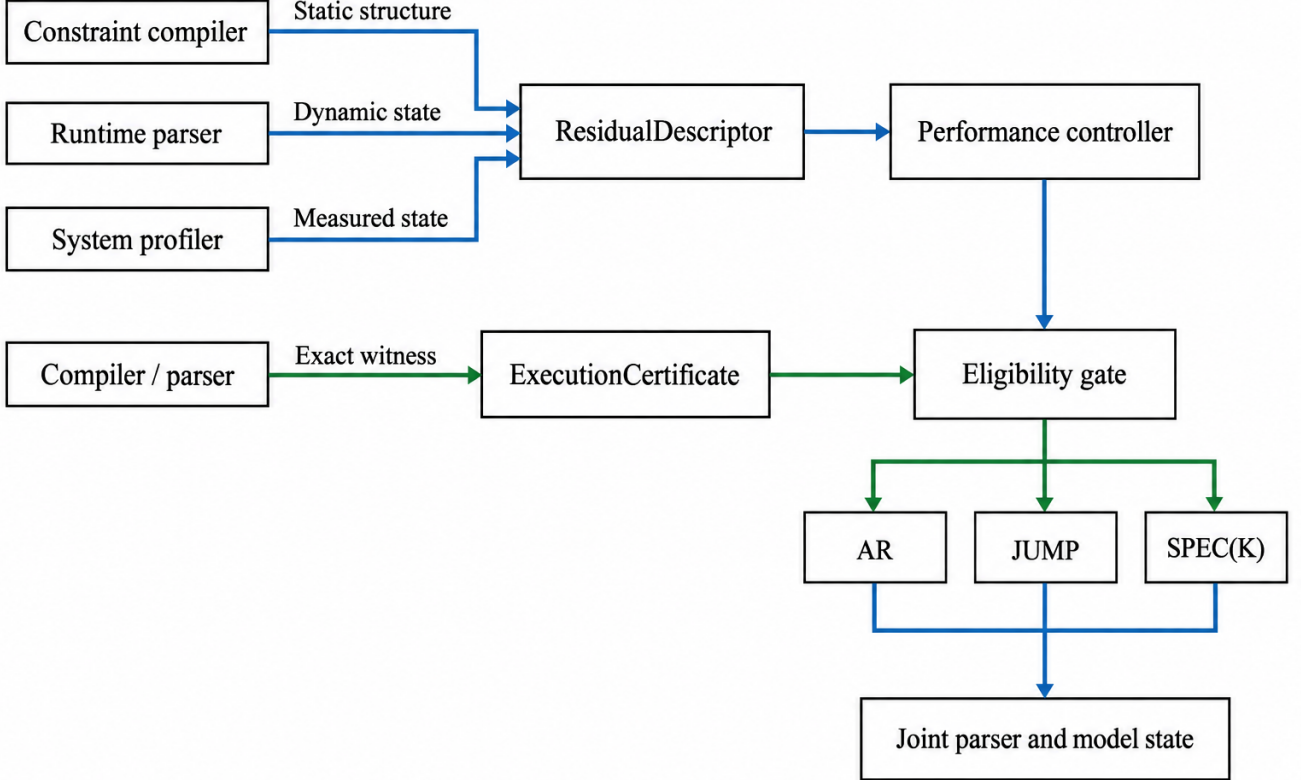


Figure 1: Proposed MaskSpec control flow. Predictive state estimates profitability, while certificate state controls semantic eligibility.

The predictor and eligibility gate consume different evidence and have different failure consequences.

4.3 Running Example

Consider the target object

```
{"status": "approved", "score": 42, "explanation": "..."}

```

The following decision epochs are illustrative, not measured. Token IDs and legal sets depend on the tokenizer and declared processor pipeline.

1. **Forced structure.** Suppose the operational state expects the literal bytes `{"status":` and a toy tokenizer makes the next three positions singleton positive-mass token sets. Static structure reports a literal region, the parser reports one active branch, and validation produces a three-token certificate. AR, SPEC, and JUMP are eligible; the controller may choose JUMP. Semantics are unchanged because the certificate, not the descriptor, establishes probability one at each position, and the target still advances through all three token IDs.
2. **Enum choice.** At the value of `status`, suppose several token IDs begin `approved` or another legal enum. The descriptor reports a small branch set and recent high draft acceptance, but there is no singleton certificate.

JUMP is ineligible; a shallow SPEC depth is an illustrative choice. Exact acceptance and correction against freshly masked target and draft laws preserve the reference prefix even if the prediction is wrong.

3. **Numeric boundary.** Near the end of 42, legality can change after every digit, sign, decimal point, or delimiter. The descriptor reports a boundary-sensitive state and no certificate. AR is the conservative illustrative choice. It is a performance fallback, not a weaker semantic mode.
4. **Open string.** Inside `explanation`, the legal set may be broad except for escaping and termination rules. If held-out calibration predicts sustained acceptance, the controller may choose a deeper SPEC action. Each tentative prefix still receives a fresh operational mask, including escape and closing-quote transitions.
5. **Tokenizer ambiguity.** Even if the parser forces the bytes `"score"`, aliases, alternate segmentations, or a token crossing the literal boundary can make more than one token ID legal. The descriptor may predict a long forced byte runway, but certification fails and JUMP remains ineligible. AR or exact SPEC is used instead. This is the trust boundary in its smallest form: a useful prediction cannot authorize an unproved transformation.

5. Execution Kernels

AR. Reference AR computes the declared p , applies $\mathcal{A}_{\text{impl}}(s)$, checks $Z_p > 0$, samples from p_{local} , and atomically advances parser and target state. It commits one token, legal EOS, or a typed empty-support result. This kernel defines the semantics to which the others are compared.

JUMP(r). JUMP is eligible only when a certificate matches the current exact state and configuration, each certified token has positive mass under the declared pre-mask distribution at its position, and batched target advancement is semantically equivalent to serial processing. Validation occurs inside a joint transaction. Failure rolls back and returns `NOT_APPLICABLE`; success advances parser and target KV/history through the certified IDs, stopping at EOS, and atomically commits that block. For a transformer, advancement normally means an extend/prefill operation that materializes KV state. JUMP reduces serial decode and synchronization rounds; it does not eliminate target work.

Prior truncation creates a real restriction: the compiler cannot certify the model-dependent condition $p(\tau_i | h_i) > 0$ at later positions without corresponding target distributions. A backend may validate all positions from batched logits before commit, but that still incurs target work. Otherwise JUMP must be disabled when truncation can zero the certified path, or all kernels must adopt and declare a different mask-before-truncation reference pipeline.

SPEC(K). Starting from a joint checkpoint, the draft proposes up to K tokens sequentially from q_{local} , recomputing the operational mask and normalizer at every tentative prefix. Proposal X_i is accepted with

$$a_i(X_i) = \min \left(1, \frac{p_{\text{local},i}(X_i)}{q_{\text{local},i}(X_i)} \right).$$

At the first rejection, SPEC samples a correction from

$$r_i(x) = \frac{[p_{\text{local},i}(x) - q_{\text{local},i}(x)]_+}{\sum_{y \in V} [p_{\text{local},i}(y) - q_{\text{local},i}(y)]_+}.$$

If every retained proposal is accepted, it samples one target bonus token. Accepted, corrected, or bonus EOS terminates immediately. Initial empty draft support makes SPEC inapplicable without mutation; later draft exhaustion verifies a shorter round. Empty target support returns the same ordered committed-prefix/failure result as sequential AR. Parser, target, and draft state publish exactly the accepted/corrected/bonus prefix and roll back every other suffix.

The initial design requires shared token IDs and constraint semantics between draft and target. Lossless heterogeneous-vocabulary speculation exists Timor et al. 2025, but importing it needs a separate proof. Floating-point underflow, approximate normalization, and nondeterministic device reductions can make mathematically exact sampling approximate in execution.

Full proposed pseudocode appears in Appendix A.

6. Correctness Argument

The propositions below describe abstract kernels. Their status labels distinguish abstract mathematical arguments from implementation obligations. Appendix B preserves the complete assumptions, derivations, counterexamples,

invariants, and test obligations.

Proposition 1 (constrained speculative prefix kernel; proof sketch under stated assumptions). Assume every tentative prefix uses the reference processors and $\mathcal{A}_{\text{impl}}$; target and draft support exist at required draws; token IDs and EOS semantics agree; auxiliary random streams are conditionally independent of future reference draws given the tentative prefix; residual correction is exact; and rollback restores the joint accepted-prefix state. Then SPEC emits a random finite block $B = (B_1, \dots, B_L)$ that can be coupled to $Y_{1:L}$, a stopped prefix of sequential reference-AR samples. If target support first fails after j committed tokens, both processes return the same prefix and typed failure. The statement does not condition on L , which may correlate with token values. The one-step acceptance mass is $\min(p_{\text{local}}(x), q_{\text{local}}(x))$; residual correction supplies the remaining target mass. Induction at freshly masked prefixes gives the stopped-prefix result across rejection, all-accepted bonus, draft exhaustion, and EOS branches. A complete measure-theoretic construction of the variable-length kernel remains an open proof obligation.

Proposition 2 (rollback observational equivalence; specification-level obligation). If a checkpoint captures every field affecting semantic observations, handles obey the transaction contract, and **advance** has no irreversible semantic side effect, then **checkpoint**; **advance**(z); **rollback** is observationally equivalent to the checkpoint state. This remains a representation-level proof obligation across every MaskForge route. Differential rollback-versus-replay testing, not state-hash equality alone, is the acceptance criterion.

Proposition 3 (singleton-certified JUMP; proof sketch under stated assumptions). If $\mathcal{A}_{\text{impl}}(s_i) = \{\tau_i\}$, $p(\tau_i \mid h_i) > 0$, transitions and EOS semantics match reference AR, and batched target advancement is algorithmically equivalent to serial advancement under the declared numerical contract, then JUMP preserves the constrained token law over the certified path and produces the post-state required by later AR. Singleton legality plus positive mass makes each local probability one; induction forces the token sequence. Equal bytes alone do not satisfy the premise. Algorithmic distributional equivalence does not imply bitwise equality between prefill and serial device kernels.

Proposition 4 (adapted composition; proof sketch under stated assumptions). Let $T_0 = 0$ and $T_{n+1} = T_n + L_n$, where action n commits $L_n \geq 1$ tokens unless it returns a typed terminal result. Require T_n and each shifted block length to be stopping times, certificate probing and action choice to use only $\mathcal{F}_{T_n}$ and $\mathcal{G}_n$, and every selected kernel to emit a stopped reference prefix conditional on that information. Require observational rollback, absorbing EOS, and atomic timeout, cancellation, retry, and partial-publication semantics. Then any history-dependent composition preserves every finite-cylinder probability of the local reference process; under almost-sure reference termination and matching terminal types, the stopped token-sequence laws agree. The sketch inducts over decision epochs: an adapted mixture cannot condition on future proposal or target randomness, and concatenated stopped reference prefixes preserve cylinder probabilities across action boundaries. Formalizing interruption events in the probability space remains open.

Proposition 5 (finite-action estimation; proved under a fully specified abstract model). For a finite action set, if $|\hat{U}(a) - U(a)| \leq \varepsilon$ for every action and $\hat{a}$ maximizes $\hat{U}$, then $U(a^*) - U(\hat{a}) \leq 2\varepsilon$, where a^* maximizes true utility. This is a pointwise decision-error result, not a generalization or long-horizon regret theorem.

Proposition 6 (termination preservation; proof sketch under stated assumptions). If EOS identity and legality are shared, no kernel commits beyond EOS, empty support has reference-AR typing, and the reference terminates almost surely, Propositions 1, 3, and 4 imply equality of stopped sequence laws. Without almost-sure termination, the conclusion is limited to finite-prefix laws plus equal nontermination mass.

7. Cost Model and Adaptive Depth

Let $L_K \in \{0, \dots, K+1\}$ be a SPEC round’s committed length, including zero progress on a pre-emission typed result. The implementation-facing expectation is the tail sum

$$G(K) = \mathbb{E}[L_K] = \sum_{m=1}^{K+1} \Pr(L_K \geq m).$$

Under the simplified accepted-prefix-plus-one algorithm, additionally assume positive draft and target support through depth K , no terminating EOS, and conditional acceptance probabilities α_i . With

$$P_i = \prod_{j=1}^i \alpha_j,$$

the expected progress is

$$G(K) = 1 + \sum_{i=1}^K P_i.$$

For constant α ,

$$G(K) = \begin{cases} \frac{1 - \alpha^{K+1}}{1 - \alpha}, & \alpha \neq 1, \\ K + 1, & \alpha = 1. \end{cases}$$

EOS, draft exhaustion, or empty target support require the tail-sum form or direct trace estimates; acceptance rate alone is insufficient. Let C_K be expected end-to-end SPEC cost and $\Delta C_{K+1} > 0$. Holding earlier predictions fixed, adding depth improves the simplified throughput ratio exactly when

$$\frac{G(K) + P_{K+1}}{C_K + \Delta C_{K+1}} > \frac{G(K)}{C_K} \iff \frac{P_{K+1}}{\Delta C_{K+1}} > \frac{G(K)}{C_K}.$$

In general replace P_{K+1} with ΔG_{K+1} . For net utility $U_K = G(K) - \lambda C_K$, the condition is $\Delta G_{K+1} > \lambda \Delta C_{K+1}$. A threshold in K follows only under a single-crossing condition such as nonincreasing marginal progress and nondecreasing marginal cost. Backend batching, launch thresholds, cache layout, and state dependence can violate it, so MaskSpec scores a finite backend-specific set rather than claiming a universal depth.

JUMP is profitable only when serial execution costs more than certification plus batched advancement:

$$C_{\text{serial-AR}}(r) > C_{\text{cert}}(r) + C_{\text{target-prefill}}(r) + C_{\text{constraint-advance}}(r) + C_{\text{KV}}(r) + C_{\text{sync}}(r).$$

A failed late validation also charges target validation, rollback, and AR fallback. Sound certificates can therefore be useless or harmful when coverage is low or target prefill dominates.

Costs are an accounting decomposition, not an additive wall-clock identity. For each action report CPU, accelerator, memory, and synchronization work, plus a dependency DAG whose longest weighted path gives C_{wall} . In general,

$$C_{\text{wall}}(a) \neq \sum_i C_i(a),$$

because parsing, device work, transfers, and launch preparation can overlap. Component labels must not double-count work already inside target or draft kernels. Compilation, request setup, decision-epoch overhead, and per-token cost are reported separately. Batch, cache warmth, precision, KV layout, and memory bandwidth condition the measurements.

The default policy invokes descriptor construction and selection once per decision epoch; certificate discovery is a separately charged probe. Any alternative cadence must be frozen and charged at its actual invocation points. A myopic ratio objective estimates $\mathbb{E}[G \mid o] / \mathbb{E}[C_{\text{wall}} \mid o]$, not $\mathbb{E}[G/C \mid o]$. A net objective avoids this distinction but requires a tuning-only backend-specific λ . Neither objective automatically optimizes the full remaining trajectory.

7.1 Semi-Markov Control View

Variable-length actions make MaskSpec a semi-Markov, or renewal-style, control problem. At epoch n , the controller observes O_n , samples $A_n \sim \pi(\cdot \mid O_n)$, incurs random wall duration C_n , and commits random length L_n . The next token-indexed decision epoch is

$$T_{n+1} = T_n + L_n.$$

The policy terminates on EOS or the reference process's typed failure. Four distinct objectives are useful:

$$\frac{\mathbb{E}[L_n \mid O_n, A_n]}{\mathbb{E}[C_n \mid O_n, A_n]}, \quad \frac{\mathbb{E}[C_n \mid O_n, A_n]}{\mathbb{E}[L_n \mid O_n, A_n]},$$

$$\mathbb{E}[L_n - \lambda C_n \mid O_n, A_n], \quad \min_{\pi} \mathbb{E}_{\pi} \left[\sum_{n < N_{\text{term}}} C_n \right].$$

The first two are immediate throughput and latency-per-token criteria, the third is immediate net utility, and the fourth is expected full-generation latency. Optimizing an immediate ratio does not generally optimize the trajectory because an action changes the next epoch, KV layout, controller cadence, and future action set. The first MaskSpec controller is therefore a practical myopic approximation, not a globally optimal policy.

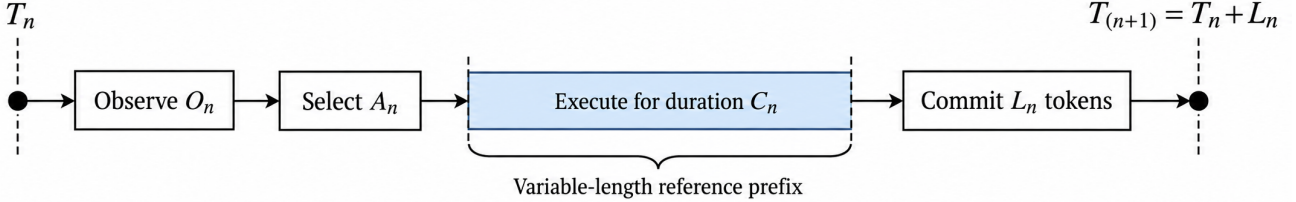


Figure 2: Proposed decision-epoch timeline for a variable-length MaskSpec action.

Private speculative work is not controller-visible future information.

8. Pre-Registered Evaluation

The evaluation asks whether MaskSpec is correct, useful, and worth its overhead. Each data point is one complete generation request, not one token. Policies run on paired prompts and constraints. Tuning and test sets are separated by schema or grammar family, and every threshold is frozen before the held-out test.

Hypothesis	Primary comparison	Evidence required	Failure outcome
H1: residual state adds predictive value	Full descriptor versus mask density and model signals	Lower paired action regret after feature cost	Drop unhelpful fields; narrow “compiler-guided” if static features add no value
H2: adaptive scheduling helps	AR/JUMP/SPEC(K) policy versus the best fixed exact policy chosen on tuning data	Practically meaningful paired latency reduction	Prefer the tuned fixed policy
H3: gains survive full accounting	End-to-end critical path versus kernel-only timing	Benefit remains after all parser, certificate, model, KV, controller, rollback, and synchronization costs	Reject the end-to-end speed claim and identify the hidden cost
H4: JUMP is safe and useful	Certified JUMP versus serial constrained AR	No detected law violation, useful certificate coverage, and lower paired latency after failed probes are charged	Disable JUMP on any semantic failure; retain it only as a soundness result if it has no speed benefit
H5: transactions repay their cost	Checkpoint/rollback versus replay and full cloning	Lower paired latency within a fixed memory budget	Keep the transaction contract but use replay or cloning

Correctness gate. Performance results are reported only after exhaustive tiny-model comparisons and rollback-versus-replay tests pass. The cases include rejection at every speculative position, fresh masks at every tentative prefix, empty support, EOS, token aliases, alternate segmentation, boundary-crossing tokens, UTF-8 fragments,

escapes, numbers, recursion, unions, dynamic references, stale handles, cancellation, and every MaskForge route. Zero failures in tests are evidence, not proof.

Workloads and baselines. Workloads use pinned versions of JSONSchemaBench, the JSON Schema Draft 2020-12 Test Suite, regular and grammar constraints, and adversarial tokenizer cases. JSONSchemaBench contains 9,558 schemas from ten sources and mixes dialects Geng et al. 2025; each input retains its declared dialect. Baselines include MaskForge constrained AR, fixed- K SPEC, the best fixed exact action, mask-density-only and model-only controllers, reproducible DOMINO-style behavior, AWRS when its predicate matches $\mathcal{A}_{\text{impl}}$, and maintained engines on a common supported subset. Unconstrained decoding is a cost reference, not a semantics-equivalent baseline.

Oracle and measurements. The primary oracle is myopic: at one checkpoint it selects the best immediate action without future acceptance or EOS information. A trajectory oracle needs a complete dynamic model; a clairvoyant oracle is labeled only as an unattainable upper bound. Common random numbers do not make counterfactual trajectories identical when actions change block boundaries, RNG use, KV layout, or later choices. Measurements include end-to-end latency, throughput, tail latency, acceptance and rejection position, selected actions, certificate coverage, fallback, memory, and the critical-path time of every component.

Decision rule. Runs use fixed seeds, warmups, repeated trials, and randomized policy order. Hierarchical bootstrap intervals account for schema-family dependence, with multiplicity control across the five primary hypotheses. A result must show both statistical uncertainty and a minimum practically important effect. That margin is chosen before evaluation from deployment requirements; this proposal does not invent one. Raw traces retain failures and timeouts and regenerate every result. The local Ryzen 3 5300U host supports CPU development only; accelerator conclusions require named devices, software versions, precision, batching, and memory policy.

9. Limitations, Scope, and Conclusion

Threats to validity. Output semantics differ across engines; validity rate cannot prove a distributional law. Asynchrony, caching, compilation amortization, placement, frequency scaling, and policy order can create false speedups. Tokens and prompts sharing schemas are dependent, so per-token pseudo-replication is invalid. One model pair, tokenizer, benchmark, or CPU cannot establish GPU-serving behavior. Descriptors may memorize schema families, certificate coverage may be workload-specific, and a bounded literature search cannot establish universal absence.

Open obligations. A future implementation must prove route-complete rollback equivalence, define EOS across tokenizer/model/constraint layers, test aliases and boundary crossings, establish target-prefill equivalence to serial advancement under its numerical model, analyze finite-precision rejection correction, and measure descriptor and certificate cost. Byte-equivalence certificates broader than singleton token paths remain open. Cancellation must be atomic or explicitly represented in the reference law; publishing a tentative block, retrying only slow branches, or dropping timeouts from correctness analysis can invalidate composition.

Scope and status. MaskSpec remains a pre-registered design: the transactions, certificates, kernels, controller, and experiments specified here have not been implemented or evaluated, and no speedup, coverage, controller advantage, or system-level exactness result is claimed. The abstract propositions are conditional on their explicit premises. Applications requiring the globally grammar-conditioned law need a different reference process and new kernel proofs; the initial action set is not exhaustive, and AWRS may outperform it.

Reproducibility. A future artifact will pin the MaskForge, model, tokenizer, benchmark, schema-suite, backend, and driver revisions; publish processor order, seeds, splits, dialect transformations, supported-vocabulary matrices, controller parameters, typed failures, and raw event traces; and include a slow reference decoder, exact toy distributions, transaction traces, certificate counterexamples, and deterministic regeneration commands.

MaskSpec’s proposed contribution is a disciplined interface between compiler knowledge and speculative execution. Residual state predicts which action may be useful; certificate state proves which transformation is allowed. Under the declared local reference law, a singleton-positive-mass JUMP, an exact stopped-prefix SPEC kernel, and an adapted scheduler can compose without changing token-sequence probabilities while still permitting performance mistakes to fall back safely to AR. The remaining research question is empirical and deliberately falsifiable: whether that semantic structure exposes enough reusable runway and predictive signal to repay its full end-to-end cost.

Appendix A. Full Proposed Algorithms

Algorithm 1: Reference Locally Constrained AR

Inputs: committed history h , constraint state s , configured pipeline, and target-sampling RNG. **Mutable state:** joint parser/target transaction. **Precondition:** joint state represents (h, s) . **Output:** one committed token, legal EOS, or typed empty-support failure. Failure before sampling leaves state unchanged.

```
REFERENCE-AR(h, s)
  p <- apply the declared non-constraint pipeline to target logits at h
  A <- operational-mask(s)
  Z <- sum_x p[x] * 1[x in A]
  if Z = 0: return EMPTY_TARGET_SUPPORT without mutation
  x <- sample(p[x] * 1[x in A] / Z)
  atomically advance target history and constraint state by x
  if x = EOS: return TERMINAL and prohibit later tokens
  return COMMITTED(x)
```

Algorithm 2: Singleton Certification and JUMP

Inputs: state s_1 , exact generation/configuration identity v , and $R_{\max}$. **Mutable state:** temporary parser transaction. **Output:** nonempty certificate or NOT_CERTIFIABLE; exploration always rolls back.

```
CERTIFY-SINGLETON(s_1, v, R_max)
  m <- parser.checkpoint()
  path <- empty
  for i = 1, ..., R_max:
    A_i <- operational-mask(current parser state)
    if A_i is not the singleton {tau_i}: break
    append (exact state identity, tau_i, successor witness) to path
    if parser.advance(tau_i) fails:
      parser.rollback(m); return CERTIFICATION_ERROR
    if tau_i = EOS: break
  parser.rollback(m)
  if path is empty: return NOT_CERTIFIABLE
  return certificate(path, exact start-state identity, v, EOS policy)
```

Validation rechecks exact state identities and singleton witnesses or invokes a formally equivalent compiled validator. Later JUMP validation uses a distinct joint transaction. On a mismatch, nonpositive target mass, or target/parser failure, it restores all validation-time mutations and returns NOT_APPLICABLE. On success, it publishes the target and parser state exactly through the certified block and stops at EOS.

Algorithm 3: Exact Constrained SPEC(K)

Inputs: committed (h, s) , $K \geq 1$, target and draft pipelines, and disjoint proposal, acceptance, and target-sampling RNG streams. **Mutable state:** joint parser/target/draft transaction with private tentative prefix states. **Preconditions:** shared token IDs and constraint/EOS semantics, exact rollback, and fresh masks/normalizers at every prefix. **Output:** nonempty committed block, EOS, ordered committed-prefix/failure, or NOT_APPLICABLE with unchanged state.

```
CONSTRAINED-SPEC(h, s, K)
  root <- joint.checkpoint()
  create a private parser/draft branch rooted at root
  sequentially propose y_1, ..., y_K from q_local on that branch
  record a private prefix checkpoint after each y_i
  if q has empty support before y_1:
    joint.rollback(root); return NOT_APPLICABLE
  stop proposal generation immediately if EOS is proposed
  evaluate p_local at every retained prefix and stage aligned target KV
  for i over proposed tokens:
    if target support at i is empty:
      restore root; publish only the accepted-prefix joint state
      return (COMMITTED(earlier prefix), EMPTY_TARGET_SUPPORT)
    if acceptance_uniform_i <= min(1, p_local_i[y_i] / q_local_i[y_i]):
```

```

    append y_i to accepted prefix
    if y_i = EOS:
        restore root; publish state exactly through EOS; return TERMINAL
    else:
        sample correction c from normalized [p_local_i - q_local_i]_+
        restore root; publish staged state through accepted prefix
        advance parser, target, and draft/recomputed draft by c
        atomically commit accepted prefix plus c
        if c = EOS: return TERMINAL
        return COMMITTED(block)
if target support for bonus is empty:
    restore root; publish and commit accepted proposal prefix
    return (COMMITTED(accepted prefix), EMPTY_TARGET_SUPPORT)
sample bonus b directly from p_local at the next prefix
restore root; publish staged state through accepted proposal prefix
advance parser, target, and draft/recomputed draft by b
atomically commit accepted proposal prefix plus b
if b = EOS: return TERMINAL
return COMMITTED(block)

```

If draft support ends after a nonempty prefix, verify that shorter prefix and apply the same correction/bonus rule. “Restore root; publish through prefix” is one atomic specification-level operation. An implementation may reuse staged KV pages and parser checkpoints or replay, but it must expose the same post-state.

Algorithm 4: Adaptive MaskSpec Scheduling

Inputs: finite backend-specific action set, certificate validator, and logically separate controller, proposal, acceptance, and target RNG streams. **Mutable state:** committed parser/target/draft transaction and calibrated controller state. **Output:** the same stopped token law as reference AR under Proposition 4.

```

MASKSPEC-DECODE(initial state)
while not terminal:
    o <- charged descriptor fields available at this epoch
    candidates <- {AR} union {SPEC(K): K in configured set}
    cert_result <- NOT_ATTEMPTED
    probe <- policy.should_attempt_certificate(o, controller RNG)
    if probe:
        cert_result <- charged CERTIFY-SINGLETON from parser checkpoint
        if cert_result is valid: add JUMP(r, cert_result)
    a <- policy(o, candidates, cert_result, past profiler, controller RNG)
        without inspecting future kernel randomness
    result <- execute a at one joint checkpoint
    if result = NOT_APPLICABLE: execute AR from unchanged checkpoint
    if result is typed failure or EOS: return result
    publish only committed-block measurements to future epochs

```

Appendix B. Full Proofs and Proof Obligations

Proposition 1: Constrained Speculative Prefix Kernel

Status. Proof sketch under stated assumptions. A complete measure-theoretic construction of the variable-length kernel and its interruption semantics, plus finite-precision realization, remain open.

Let a round emit random block $B = (B_1, \dots, B_L)$, where L depends on rejection, depth, draft exhaustion, EOS, and target failure. Enlarge the within-round filtration to reveal proposal draws, acceptance uniforms, correction/bonus draws, and emitted tokens in order. Let $Y_1, Y_2, \dots$ be sequential reference-AR samples augmented with the same independent auxiliary proposal and acceptance randomness.

Assumptions. Every tentative prefix uses $\mathcal{A}_{\text{impl}}$ and the declared processors; $Z_p > 0$ at each required reference draw and $Z_q > 0$ wherever the draft samples; draft and target share token IDs and EOS semantics; proposal, acceptance, and target-sampling streams have the declared conditional independence; Algorithm 3 uses the residual correction; and rollback restores the joint accepted-prefix state.

Conclusion. While target support remains positive, there is a coupling under which $B = Y_{1:L}$, with $L \geq 1$ unless a typed failure occurs before emission, and L is a stopping time in the enlarged filtration. This does not assert that $B \mid L = m$ has the unconditional m -token reference distribution. If target support first becomes empty after j emitted tokens, Algorithm 3 returns the same ordered pair: a reference prefix of length j , followed by failure at $j + 1$, as sequential AR.

Proof. At one tentative prefix, for proposal $D \sim q_{\text{local}}$,

$$\Pr(\text{accepted output} = x) = q_{\text{local}}(x) \min\left(1, \frac{p_{\text{local}}(x)}{q_{\text{local}}(x)}\right) = \min(p_{\text{local}}(x), q_{\text{local}}(x)).$$

The rejection probability is

$$R = 1 - \sum_x \min(p_{\text{local}}(x), q_{\text{local}}(x)) = \sum_x [p_{\text{local}}(x) - q_{\text{local}}(x)]_+.$$

The accepted contribution is zero for tokens with $q_{\text{local}}(x) = 0$; target-only mass is supplied by correction. The rejection branch contributes

$$Rr(x) = [p_{\text{local}}(x) - q_{\text{local}}(x)]_+,$$

so accepted and corrected mass sum to $p_{\text{local}}(x)$. Conditional on an accepted emitted prefix, repeat with freshly computed masks, p , and q . The first rejection emits its correction and stops; an all-accepted chain receives a direct target bonus; any accepted, corrected, or bonus EOS is emitted and stops immediately. Induction therefore couples every emitted position to the next reference sample, and $\{L \leq i\}$ is decidable from information revealed through position i . Empty target support follows the reference failure branch. Empty initial draft support returns NOT_APPLICABLE before mutation; later exhaustion invokes the shorter-round rule.

Invariant and test. Every verifier mask and normalizer equals reference replay at the same prefix. Exhaustive tiny vocabularies exercise rejection at every position, proposal-only and target-only support, EOS, bonus, and empty-support cases.

Failure modes. Stale masks, inconsistent processor order, approximate acceptance/correction, undeclared RNG reuse, tokenizer mismatch, partial rollback, inconsistent EOS, and finite precision.

Proposition 2: Rollback Observational Equivalence

Status. Specification-level obligation.

Assumptions. A checkpoint captures every field affecting semantic observation; handles obey Section 4.2; **advance** has no irreversible semantic side effect; and persistent telemetry is external, noninterfering, and disclosed to the controller filtration before use.

Conclusion. For valid mark m , **checkpoint**; **advance**(z); **rollback**(m) leaves the same mask, accepted bytes, descriptor, lifecycle/error state, handle validity, and future transitions as state m . External telemetry may record tentative work but cannot distinguish the restored session through any semantic operation.

Proof obligation. A representation proof must enumerate every mutable field for regular, structured, and hybrid routes and classify it as transactional semantic state or external telemetry. An omitted mask-affecting cache is a counterexample. Differential rollback-versus-replay over nested traces, invalid handles, reset, commits, and every route is required; state hashes are diagnostic only.

Proposition 3: Singleton-Certified JUMP

Status. Proof sketch under stated assumptions at the abstract token-model level.

Assumptions. For each certified position, $\mathcal{A}_{\text{impl}}(s_i) = \{\tau_i\}$ and $p(\tau_i \mid h_i) > 0$; transitions, EOS, and special-token semantics match reference AR; and batched target advancement is semantically equivalent to serial processing.

Conclusion. JUMP(r) preserves the constrained token law for r positions and leaves the target/constraint post-state needed by subsequent AR.

Proof. Singleton legality and positive mass give $p_{\text{local}}(\tau_i \mid h_i, s_i) = 1$. Induction forces the certified token path. Equivalent parser and target advancement produces the same reference post-state. The conclusion fails for a common byte prefix without singleton token IDs, for a zero-mass singleton after truncation, or when batched advancement changes the declared numerical semantics.

Counterexamples and tests. Reject certificates for aliases; $[ab]$ versus $[a, b]$; boundary-crossing tokens; UTF-8 fragments; normalized text with distinct IDs; early EOS; empty-byte special IDs; stale versions; and distinct semantic states sharing a cache hash. Compare final logits, mask, lifecycle, and future transitions with token-by-token replay.

Proposition 4: Adapted Composition at Variable-Length Decision Epochs

Status. Proof sketch under stated assumptions at the abstract prefix-kernel level; interruption semantics and implementation premises remain unverified.

Let $T_0 = 0$ and $T_{n+1} = T_n + L_n$. The filtration $\mathcal{H}_t$ reveals committed reference tokens, semantic states, controller draws, and only auxiliary facts exposed through token boundary t . Private proposal suffixes are not placed in $\mathcal{H}_t$ before their semantic positions even if computed early. At epoch n , an $\mathcal{F}_{T_n}$ -measurable policy may attempt certification; $\mathcal{G}_n$ adds its decision, result, charged cost, and fresh controller randomness, but no future proposal, acceptance, or target-sampling draw. Controller, proposal, acceptance, and target-sampling randomness are conceptually distinct streams.

Assumptions. T_0 is a stopping time; conditional on $\mathcal{G}_n$, L_n is a stopping time for the shifted filtration; probing is $\mathcal{F}_{T_n}$ -measurable and read-only or rolled back; action choice uses only $\mathcal{G}_n$; every selected kernel conditionally couples its block to a stopped reference prefix; $L_n \geq 1$ unless a typed terminal result occurs; rollback is observationally exact; EOS is absorbing; and deadline, cancellation, retry, or resource failure cannot choose whether to publish a block using uncommitted token-correlated values unless that interruption is part of the reference process.

Conclusion. Every history-dependent composition satisfying these conditions preserves all finite-cylinder probabilities of the locally constrained reference process. Under almost-sure reference termination and matching terminal typing, the stopped token-sequence laws are equal.

Proof. If T_n and shifted L_n are stopping times, then $T_{n+1} = T_n + L_n$ is a stopping time. Conditional on $\mathcal{G}_n$, adaptedness selects a mixture without conditioning on future kernel randomness. Each component emits a stopped reference prefix, so the policy-weighted mixture does too. Taking expectations over probe/controller randomness and concatenating the block establishes the induction hypothesis at T_{n+1} . Iterated conditioning gives equality for every finite cylinder, including cylinders crossing action boundaries. Almost-sure termination and absorbing EOS identify the stopped laws. The proof never conditions on realized block length alone.

Counterexample and test. A controller that observes whether a proposal will be rejected before selecting AR or SPEC can bias output. Log every field read before selection, keep counterfactual information offline, and exhaustively enumerate finite-cylinder probabilities for deterministic and independently randomized policies on toy models.

Proposition 5: Finite-Action Estimation Bound

Status. Proved under a fully specified abstract finite-action model.

Suppose $|\hat{U}(a) - U(a)| \leq \varepsilon$ for every action, $\hat{a} \in \arg \max_a \hat{U}(a)$, and $a^* \in \arg \max_a U(a)$. Then

$$\begin{aligned} U(a^*) - U(\hat{a}) &\leq [U(a^*) - \hat{U}(a^*)] + [\hat{U}(\hat{a}) - U(\hat{a})] \\ &\leq 2\varepsilon, \end{aligned}$$

because $\hat{U}(\hat{a}) \geq \hat{U}(a^*)$. This proves only pointwise finite-action decision error.

Proposition 6: Termination Preservation

Status. Proof sketch under stated assumptions.

Assumptions. EOS is represented consistently in V ; $\mathcal{A}_{\text{impl}}$ admits EOS exactly when termination is legal; no kernel commits beyond EOS; empty support has the same typed result as AR; and the reference terminates almost surely, or equality is claimed only on finite prefixes.

Conclusion and proof. Propositions 1, 3, and 4 give equality for every finite cylinder, including cylinders ending at EOS. The shared no-post-EOS rule makes termination absorbing, so the stopped laws agree. Without almost-sure termination, equality is limited to finite-prefix laws and nontermination mass.

References

1. Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. “Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation.” *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:3658–3673, 2024. <https://proceedings.mlr.press/v235/beurer-kellner24a.html>
2. Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. “Accelerating Large Language Model Decoding with Speculative Sampling.” arXiv:2302.01318, 2023. <https://arxiv.org/abs/2302.01318>
3. Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. “Sequoia: Scalable and Robust Speculative Decoding.” *Advances in Neural Information Processing Systems 37*, 2024. DOI: 10.52202/079017-4116. https://proceedings.neurips.cc/paper_files/paper/2024/hash/ea1f5f0878d43ff4fb8bf64ef4a2326c-Abstract-Conference.html
4. Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ziyi Xu, Yilong Zhao, Ruihang Lai, and Tianqi Chen. “XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models.” *Proceedings of Machine Learning and Systems 7*, 2025. https://proceedings.mlsys.org/paper_files/paper/2025/hash/5c20ca4b0b20b0bd2f1d839dc605e70f-Abstract-Conference.html
5. Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. “JSONSchemaBench: A Rigorous Benchmark of Structured Outputs for Language Models.” arXiv:2501.10868v3, 2025. <https://arxiv.org/abs/2501.10868>
6. Kaixuan Huang, Xudong Guo, and Mengdi Wang. “SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths.” *Conference on Language Modeling*, 2025. <https://openreview.net/forum?id=Y131N9fUbU>
7. Yaniv Leviathan, Matan Kalman, and Yossi Matias. “Fast Inference from Transformers via Speculative Decoding.” *Proceedings of the 40th International Conference on Machine Learning*, PMLR 202:19274–19286, 2023. <https://proceedings.mlr.press/v202/leviathan23a.html>
8. Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Gaurav Jain, Oren Pereg, Moshe Wasserblat, and David Harel. “Accelerating LLM Inference with Lossless Speculative Decoding Algorithms for Heterogeneous Vocabularies.” *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267:59598–59620, 2025. <https://proceedings.mlr.press/v267/timor25a.html>
9. Yunlong Hou, Fengzhuo Zhang, Cunxiao Du, Xuan Zhang, Jiachun Pan, Tianyu Pang, Chao Du, Vincent Y. F. Tan, and Zhuoran Yang. “BanditSpec: Adaptive Speculative Decoding via Bandit Algorithms.” *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267:24045–24079, 2025. <https://proceedings.mlr.press/v267/hou25h.html>
10. Jonathan Mamou, Oren Pereg, Daniel Korat, Moshe Berchansky, Nadav Timor, Moshe Wasserblat, and Roy Schwartz. “Dynamic Speculation Lookahead Accelerates Speculative Decoding of Large Language Models.” *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, PMLR 262:456–467, 2024. <https://proceedings.mlr.press/v262/mamou24a.html>
11. Wenhua Nie, Zijie Meng, Kun Zou, Zheng Lin, Ziwei Li, Haoran Zheng, Jyh-Shing Roger Jang, and Hao Zhang. “Future Validity is the Missing Statistic: From Impossibility to -Estimation for Grammar-Faithful Speculative Decoding.” arXiv:2605.07698, 2026. <https://arxiv.org/abs/2605.07698>
12. Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D’Antoni. “Grammar-Aligned Decoding.” *Advances in Neural Information Processing Systems 37*, 2024. https://proceedings.neurips.cc/paper_files/paper/2024/hash/2bdc2267c3d7d01523e2e17ac0a754f3-Abstract-Conference.html
13. Brandon T. Willard and Rémi Louf. “Efficient Guided Generation for Large Language Models.” arXiv:2307.09702, 2023. <https://arxiv.org/abs/2307.09702>
14. İşıl Özgü, Yaoxuan Wu, Guy Van den Broeck, and Miryung Kim. “The Parser Already Knows: Lightweight Bias Correction in Constrained Decoding.” arXiv:2608.10137, 2026. <https://arxiv.org/abs/2608.10137>
15. Yongmin Li, Yihong Dong, Jia Li, and Ge Li. “Efficient Grammar-Constrained Decoding via Parser Stack Classification.” arXiv:2608.03065, 2026. <https://arxiv.org/abs/2608.03065>
16. Xingzi Xu and Karim Bouyarmane. “Trie Automata for Constrained Decoding over Large Finite Sets.” arXiv:2608.12574, 2026. <https://arxiv.org/abs/2608.12574>
17. Kanghee Park, Timothy Zhou, and Loris D’Antoni. “Flexible and Efficient Grammar-Constrained Decoding.” *Proceedings of the 42nd International Conference on Machine Learning*, PMLR 267:48262–48275, 2025. <https://proceedings.mlr.press/v267/park25l.html>
18. Mohsen Arjmandi. “GRID: Grammar-Railed Decoding for Enterprise SQL Generation.” arXiv:2607.11951, 2026. <https://arxiv.org/abs/2607.11951>
19. Benjamin Lipkin, Benjamin LeBrun, Jacob Hoover Vigly, João Loula, David R. MacIver, Li Du, Jason Eisner, Ryan Cotterell, Vikash Mansinghka, Timothy J. O’Donnell, Alexander K. Lew, and Tim Vieira. “Fast Controlled Generation from Language Models with Adaptive Weighted Rejection Sampling.” *Conference on Language Modeling*, 2025; arXiv:2504.05410v2. <https://arxiv.org/abs/2504.05410>
20. Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. “JSON Schema: A Media Type for Describing JSON Documents.” Draft 2020-12, 2022. <https://json-schema.org/draft/2020-12/json-schema-core.html>

21. Austin Wright, Henry Andrews, and Ben Hutton. “JSON Schema Validation: A Vocabulary for Structural Validation of JSON.” Draft 2020-12, 2022. <https://json-schema.org/draft/2020-12/json-schema-validation.html>

Versioned software evidence: XGrammar v0.2.6 **GrammarMatcher**, llguidance fast-forward and C API at revision bae21e5, and MaskForge revision 9426a46.